COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	Jul 9, 2026	**DATE SUBMITTED:**	Jul 21, 2026
NAME:	Carlo Mari B. Asajar	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 1.1 [BASIC C++ PROGRAMMING]

1. PROCEDURE OUTPUT

ILO A: Create Code That Follows the Basic C++ Code Structure

For this activity, you have to demonstrate the use of a function prototype. The section on class declaration and method definition will be used for the function prototype and the function will be defined in the following method definition section after the main function.

A function prototype in c++ is a declaration of the name, parameters and return type of the function before its definition.

Note:

- The driver program must call each function.
- The definitions must be after the main function

Write a C++ code the satisfies the following:

1. Create a function that will take two numbers and display the sum.
2. Create a function that will return whether variable A is greater than variable B.
3. Create a function that will take two Boolean values and display the result of all logical operations then return true if it was a success.

My Dev-C++ Code:

```cpp
#include <iostream>
#include <iomanip>

void prompt();
void lines();
void inputs(double &firstNumber, double &secondNumber);
double computeSum(double firstNumber, double secondNumber);
bool compareTwoVariables(double A, double B);
bool logicalOperations(bool value1, bool value2);

int main() {

    double firstNumber, secondNumber;
    int option = 0;
    double inputA, inputB;

    std::cout << "================== BASIC FUNCTION PROGRAM ==================" << std::endl;

    while (option != 4) {
        lines();
        prompt();
        std::cin >> option;
        std::cout << std::endl;

        switch (option) {
            case 1: {
                inputs(firstNumber, secondNumber);
                std::cout << "The sum is " << computeSum(firstNumber, secondNumber) << std::endl;
                break;
            }

            case 2: {
                inputs(firstNumber, secondNumber);
                if (compareTwoVariables(firstNumber, secondNumber)) {
                    std::cout << "A is greater than B." << std::endl;
                } else {
                    std::cout << "A is not greater than B." << std::endl;
                }
                break;
            }

            case 3: {
                while (true) {
                    std::cout << "Enter the first Boolean value (1 = true, 0 = false): ";
                    std::cin >> inputA;

                    if (inputA == 0 || inputA == 1)
                        break;

                    std::cout << "Invalid input. Please enter only 0 or 1." << std::endl;
                }

                while (true) {
                    std::cout << "Enter the second Boolean value (1 = true, 0 = false): ";
                    std::cin >> inputB;

                    if (inputB == 0 || inputB == 1)
                        break;

                    std::cout << "Invalid input. Please enter only 0 or 1." << std::endl;
                }

                lines();
```

```
65
66                  if (logicalOperations(inputA, inputB)) {
67                      lines();
68                      std::cout << "Logical operations completed successfully." << std::endl;
69                  }
70                  break;
71              }
72
73              case 4: {
74                  lines();
75                  std::cout << "Program ended. Thank you." << std::endl;
76                  break;
77              }
78
79              default: {
80                  std::cout << "Invalid option. Please try again." << std::endl;
81                  break;
82              }
83          }
84      }
85
86      return 0;
87  }
88
89  void prompt() {
90      std::cout << "(1) Sum of Two Numbers" << std::endl;
91      std::cout << "(2) Compare Two Variables" << std::endl;
92      std::cout << "(3) Perform Logical Operations" << std::endl;
93      std::cout << "(4) Exit" << std::endl << std::endl;
94      std::cout << "Select option: ";
95  }
96
```

```
97  void lines() {
98      std::cout << std::endl;
99      for (int line = 0; line < 60; line++) {
100         std::cout << "-";
101     }
102     std::cout << std::endl;
103 }
104
105 void inputs(double &firstNumber, double &secondNumber) {
106     std::cout << "Enter the first number: ";
107     std::cin >> firstNumber;
108     std::cout << "Enter the second number: ";
109     std::cin >> secondNumber;
110 }
111
112 double computeSum(double firstNumber, double secondNumber) {
113     double sum = firstNumber + secondNumber;
114     return sum;
115 }
116
117 bool compareTwoVariables(double firstNumber, double secondNumber) {
118     return firstNumber > secondNumber;
119 }
120
121 bool logicalOperations(bool inputA, bool inputB) {
122     std::cout << "\t\t     LOGICAL OPERATIONS";
123     lines();
124     std::cout << std::left;
125     std::cout << std::setw(18) << "AND   (&&)"  << ":  " << (inputA && inputB) << std::endl;
126     std::cout << std::setw(18) << "OR    (||)"   << ":  " << (inputA || inputB) << std::endl;
127     std::cout << std::setw(18) << "NOT a (!a)"<< ":  " << (!inputA) << std::endl;
128     std::cout << std::setw(18) << "NOT b (!b)"<< ":  " << (!inputB);
129     return true;
130 }
```

Sample Output:

Part 1:

```
================= BASIC FUNCTION PROGRAM =================

------------------------------------------------------------
(1) Sum of Two Numbers
(2) Compare Two Variables
(3) Perform Logical Operations
(4) Exit

Select option: 5

Invalid option. Please try again.

------------------------------------------------------------
(1) Sum of Two Numbers
(2) Compare Two Variables
(3) Perform Logical Operations
(4) Exit

Select option: 1

Enter the first number: 25
Enter the second number: 15
The sum is 40

------------------------------------------------------------
(1) Sum of Two Numbers
(2) Compare Two Variables
(3) Perform Logical Operations
(4) Exit

Select option: 1

Enter the first number: 34.9
Enter the second number: 67.3
The sum is 102.2

------------------------------------------------------------
(1) Sum of Two Numbers
(2) Compare Two Variables
(3) Perform Logical Operations
(4) Exit

Select option: 2

Enter the first number: 20
Enter the second number: 10
A is greater than B.
```

Part 2:

```
------------------------------------------------------------
(1) Sum of Two Numbers
(2) Compare Two Variables
(3) Perform Logical Operations
(4) Exit

Select option: 2

Enter the first number: 8
Enter the second number: 15
A is not greater than B.

------------------------------------------------------------
(1) Sum of Two Numbers
(2) Compare Two Variables
(3) Perform Logical Operations
(4) Exit

Select option: 3

Enter the first Boolean value (1 = true, 0 = false): 2
Invalid input. Please enter only 0 or 1.
Enter the first Boolean value (1 = true, 0 = false): 1
Enter the second Boolean value (1 = true, 0 = false): 5
Invalid input. Please enter only 0 or 1.
Enter the second Boolean value (1 = true, 0 = false): 1

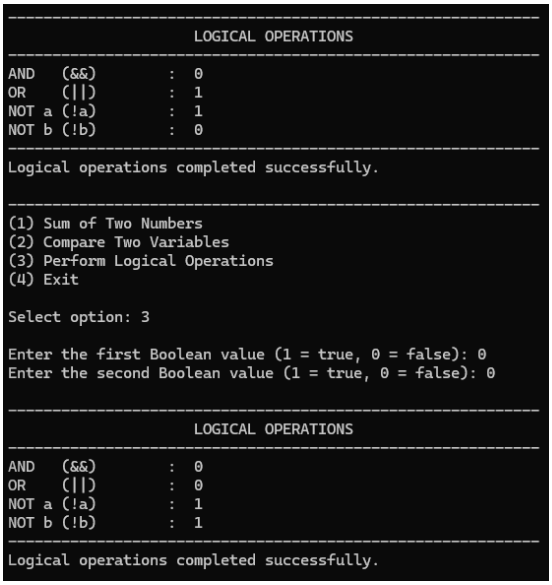
------------------------------------------------------------
                    LOGICAL OPERATIONS
------------------------------------------------------------
AND   (&&)        :  1
OR    (||)        :  1
NOT a (!a)        :  0
NOT b (!b)        :  0
------------------------------------------------------------
Logical operations completed successfully.

------------------------------------------------------------
(1) Sum of Two Numbers
(2) Compare Two Variables
(3) Perform Logical Operations
(4) Exit

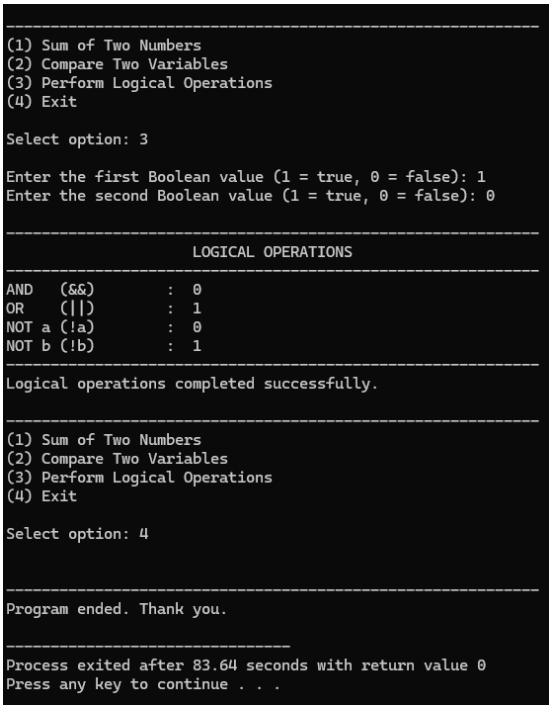
Select option: 3

Enter the first Boolean value (1 = true, 0 = false): 0
Enter the second Boolean value (1 = true, 0 = false): 1
```

Part 3:



Part 4:

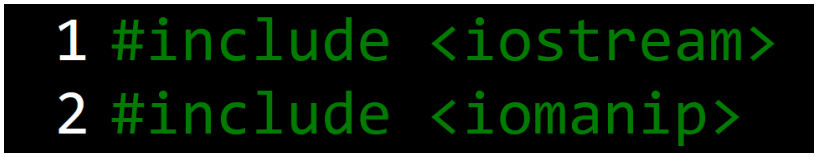
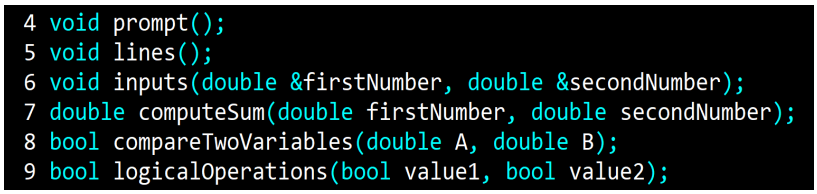
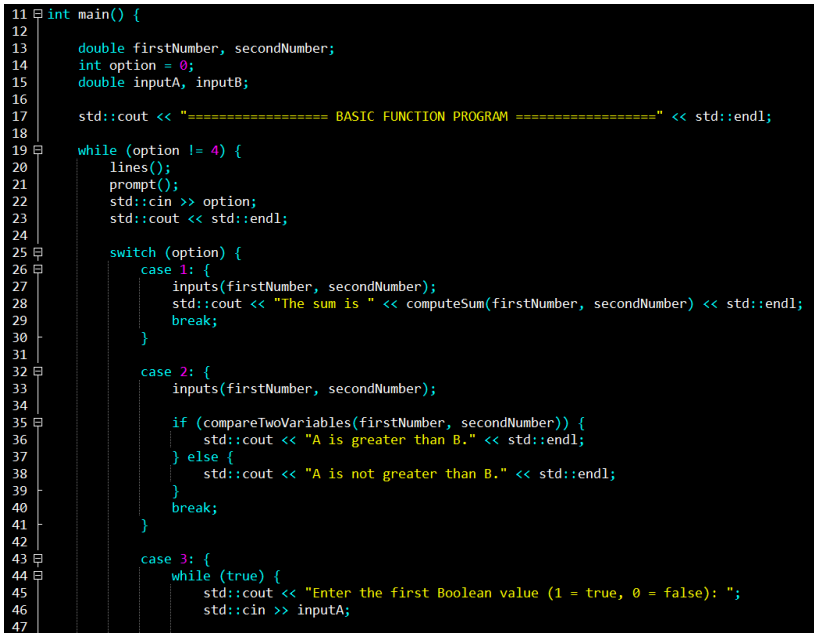


Code Analysis:

The program uses function prototypes at the beginning so every function is already declared before ***main()***, while the actual function definitions are placed after ***main()*** to follow the required C++ structure. Inside ***main()***, the program uses a ***while loop*** to keep running until option ***4*** is selected, and a ***switch statement*** is used to connect each menu option to its correct function. The ***inputs()*** function is used to collect two numbers from the user, then the ***computeSum()*** function uses the formula ***firstNumber + secondNumber*** and returns the total value that is displayed in the output. The ***compareTwoVariables()*** function is important because it uses the **> operator** to check if the first variable is greater than the second variable and returns a boolean result. For the logical operations, the program uses ***while loops*** to check if the entered values are only **0** or **1**, then the ***logicalOperations()*** function displays the results of ***AND (&&)***, ***OR (||)***, and ***NOT (!)*** operations before returning ***true*** to show that the operation was successful. The helper functions ***prompt()*** and ***lines()*** are also used to make the program output cleaner and more organized.

During the coding process, some difficulties were faced like understanding where to properly place the

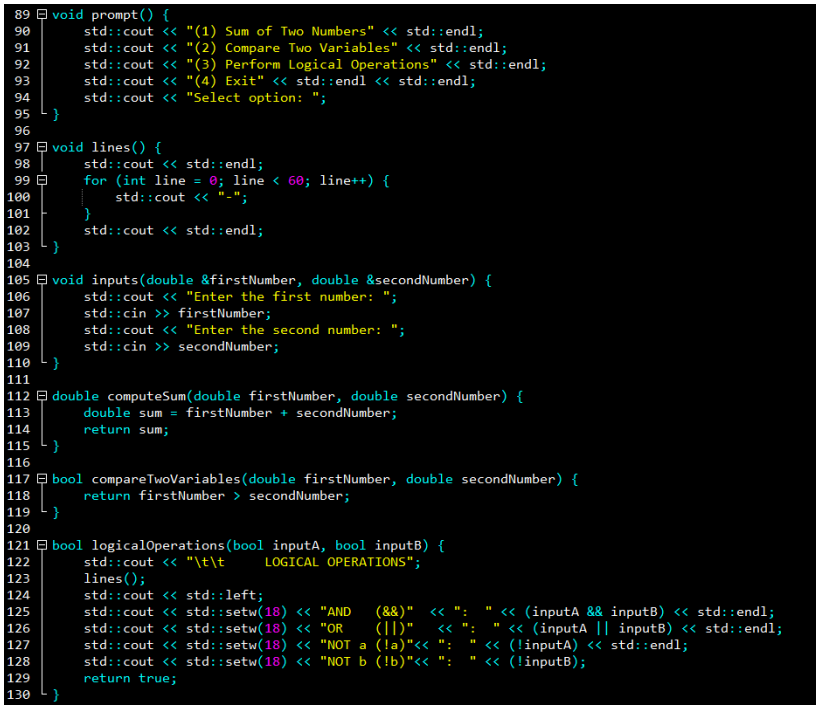
function prototypes and making sure that the function definitions stay after the ***main()*** function because it is different from the usual way of writing simple programs. Another challenge was handling the Boolean input because the program needed to accept only ***0 and 1***, so extra checking using ***while loops*** was added to prevent invalid values. There was also some difficulty in arranging the logical operation display to make the output look more readable, but using ***iomanip***, especially ***setw()***, helped fix the spacing problem.

Sections	Answer
Header File Declaration Section	
Global Declaration Section	
Class Declaration and Method Definition Section	**None**
Main Function	

```
1 #include <iostream>
2 #include <iomanip>
```

```
4 void prompt();
5 void lines();
6 void inputs(double &firstNumber, double &secondNumber);
7 double computeSum(double firstNumber, double secondNumber);
8 bool compareTwoVariables(double A, double B);
9 bool logicalOperations(bool value1, bool value2);
```

```
11 int main() {
12
13     double firstNumber, secondNumber;
14     int option = 0;
15     double inputA, inputB;
16
17     std::cout << "================= BASIC FUNCTION PROGRAM =================" << std::endl;
18
19     while (option != 4) {
20         lines();
21         prompt();
22         std::cin >> option;
23         std::cout << std::endl;
24
25         switch (option) {
26             case 1: {
27                 inputs(firstNumber, secondNumber);
28                 std::cout << "The sum is " << computeSum(firstNumber, secondNumber) << std::endl;
29                 break;
30             }
31
32             case 2: {
33                 inputs(firstNumber, secondNumber);
34
35                 if (compareTwoVariables(firstNumber, secondNumber)) {
36                     std::cout << "A is greater than B." << std::endl;
37                 } else {
38                     std::cout << "A is not greater than B." << std::endl;
39                 }
40                 break;
41             }
42
43             case 3: {
44                 while (true) {
45                     std::cout << "Enter the first Boolean value (1 = true, 0 = false): ";
46                     std::cin >> inputA;
47
```

Method Definition	

```
void prompt() {
    std::cout << "(1) Sum of Two Numbers" << std::endl;
    std::cout << "(2) Compare Two Variables" << std::endl;
    std::cout << "(3) Perform Logical Operations" << std::endl;
    std::cout << "(4) Exit" << std::endl << std::endl;
    std::cout << "Select option: ";
}

void lines() {
    std::cout << std::endl;
    for (int line = 0; line < 60; line++) {
        std::cout << "-";
    }
    std::cout << std::endl;
}

void inputs(double &firstNumber, double &secondNumber) {
    std::cout << "Enter the first number: ";
    std::cin >> firstNumber;
    std::cout << "Enter the second number: ";
    std::cin >> secondNumber;
}

double computeSum(double firstNumber, double secondNumber) {
    double sum = firstNumber + secondNumber;
    return sum;
}

bool compareTwoVariables(double firstNumber, double secondNumber) {
    return firstNumber > secondNumber;
}

bool logicalOperations(bool inputA, bool inputB) {
    std::cout << "\t\t     LOGICAL OPERATIONS";
    lines();
    std::cout << std::left;
    std::cout << std::setw(18) << "AND   (&&)"  << ":  " << (inputA && inputB) << std::endl;
    std::cout << std::setw(18) << "OR    (||)"   << ":  " << (inputA || inputB) << std::endl;
    std::cout << std::setw(18) << "NOT a (!a)"<< ":  " << (!inputA) << std::endl;
    std::cout << std::setw(18) << "NOT b (!b)"<< ":  " << (!inputB);
    return true;
}
```

ILO B: Implement Appropriate Class Definition and Instances Based on Given Requirements

In this section, the initial implementation for a class triangle will be implemented. The step-by-step procedure is shown below:

Step 1. Include the necessary header files. For this one, we only need #include <iostream>
Step 2. Create the triangle class. Assign it with private variables: totalAngle, angleA, angleB, and angleC.

```
class Triangle {
      private:
            double totalAngle, angleA, angleB, angleC;
```

Step 3. We then create public methods. The constructor must allow for creation of the object with 3 initial angles to be stored in our previously defined variables angleA, angleB and angleC. Another method has to be made if the user wants to change the initial values, this will also accept 3 arguments to change the values in angleA, angleB and angleC. Lastly, a function to validate whether the given values make our shape an actual triangle.

```
public:
            Triangle(double A, double B, double C);
            void setAngles(double A, double B, double C);
            const bool validateTriangle();

};
```

Step 4. Define the methods

```
Triangle::Triangle(double A, double B, double C) {
      angleA = A;
      angleB = B;
      angleC = C;
      totalAngle = A+B+C;
}

void Triangle::setAngles(double A, double B, double C) {
      angleA = A;
      angleB = B;
      angleC = C;
      totalAngle = A+B+C;
}

const bool Triangle::validateTriangle() {
      return (totalAngle <= 180);
}
```

Step 5. Create the driver code.

```
int main(){

      Triangle set1(40, 30, 110);
      if(set1.validateTriangle()){
            cout << "The shape is a valid triangle.\n";
      }

      else {
            cout << "The shape is NOT a valid triangle.\n";
      }
```

```
    return 0;
}
```

Include the output of running this code in section 6. Note your observations and comments.

My Dev-C++ Code:

```
#include <iostream>

class Triangle {
    private:
        double totalAngle, angleA, angleB, angleC;

    public:
        Triangle(double A, double B, double C);
        void setAngles(double A, double B, double C);
        const bool validateTriangle();

};

int main(){

    Triangle set1(40, 30, 110);
    if(set1.validateTriangle()){
        std::cout << "The shape is a valid triangle.\n";
    }

    else {
        std::cout << "The shape is NOT a valid triangle.\n";
    }

    return 0;
}

Triangle::Triangle(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}

void Triangle::setAngles(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}

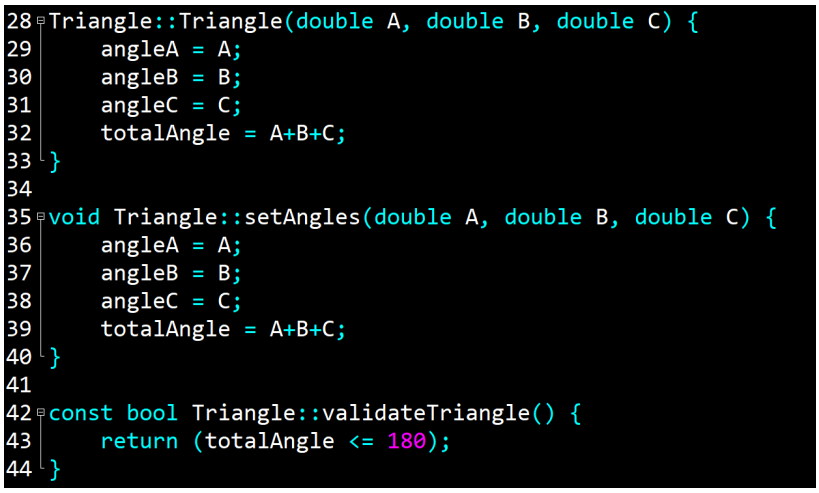
const bool Triangle::validateTriangle() {
    return (totalAngle <= 180);
}
```

Sample Output:

```
The shape is a valid triangle.

--------------------------------
Process exited after 0.7479 seconds with return value 0
Press any key to continue . . .
```

Sections	Answer
Header File Declaration Section	`#include <iostream>`
Global Declaration Section	None
Class Declaration and Method Definition Section	`class Triangle { private: double totalAngle, angleA, angleB, angleC; public: Triangle(double A, double B, double C); void setAngles(double A, double B, double C); const bool validateTriangle(); };`
Main Function	`int main(){ Triangle set1(40, 30, 110); if(set1.validateTriangle()){ std::cout << "The shape is a valid triangle.\n"; } else { std::cout << "The shape is NOT a valid triangle.\n"; } return 0; }`

Method Definition	

```
28  Triangle::Triangle(double A, double B, double C) {
29      angleA = A;
30      angleB = B;
31      angleC = C;
32      totalAngle = A+B+C;
33  }
34
35  void Triangle::setAngles(double A, double B, double C) {
36      angleA = A;
37      angleB = B;
38      angleC = C;
39      totalAngle = A+B+C;
40  }
41
42  const bool Triangle::validateTriangle() {
43      return (totalAngle <= 180);
44  }
```

Table 1-1. C++ Structure Code for Answer

The code started by including the ***#include <iostream>*** header file which is needed for using the input and output functions like ***std::cout***. The ***Triangle*** class was created with private variables ***totalAngle***, ***angleA***, ***angleB***, and ***angleC*** to store the values of the triangle angles. The public section contains the constructor ***Triangle(double A, double B, double C)***, the ***setAngles(double A, double B, double C)*** function, and the ***validateTriangle()*** function which are important parts of the program because they handle the creation, changing, and checking of the triangle values. The constructor is used to assign the starting angle values and calculate the ***totalAngle*** using ***A+B+C.*** The ***setAngles()*** function also does the same process but it is used when changing the current angle values after the object is already created. The ***validateTriangle()*** function is used to check if the total angle is valid by using the condition ***return (totalAngle <= 180)***;, where it returns true if the total angle is not more than 180 degrees. In the driver code, the object ***Triangle set1(40, 30, 110)***; was created, and the function ***set1.validateTriangle()*** was called to test if the given angles can be considered as a valid triangle. The output shows that the shape is a valid triangle because the total of the angles is ***180*** degrees.

Table 1-2. ILO B output observations and comments

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve Different Problems using the C++ Programming Language

The supplementary activities are meant to gauge your ability in using C++. The problems below range from easy to intermediate to advanced problems. Note your difficulties after answering the problems below.

1. Create a C++ program to swap the two numbers in different variables.

My Dev-C++ Code:

```cpp
#include <iostream>
#include <iomanip>

void lines();
void input(double &firstNumber, double &secondNumber);
void swapNumbers(double &firstNumber, double &secondNumber);
void prompt();

int main() {

    double firstNumber, secondNumber;
    int option = 0;

    std::cout << "===== SWAPPING TWO NUMBERS =====" << std::endl;

    do {
        lines();
        input(firstNumber, secondNumber);

        std::cout << std::endl << "\tBEFORE SWAPPING";
        lines();
        std::cout << std::left;
        std::cout << std::setw(15) << "First number"  << ":  " << firstNumber << std::endl;
        std::cout << std::setw(15) << "Second number" << ":  " << secondNumber << std::endl;

        swapNumbers(firstNumber, secondNumber);

        std::cout << std::endl << "\tAFTER SWAPPING";
        lines();
        std::cout << std::left;
        std::cout << std::setw(15) << "First number"  << ":  " << firstNumber << std::endl;
        std::cout << std::setw(15) << "Second number" << ":  " << secondNumber << std::endl;

        do {
            lines();
            prompt();
            std::cin >> option;

            if (option != 1 && option != 2) {
                std::cout << std::endl << "Invalid option. Please try again." << std::endl;
            }

        } while (option != 1 && option != 2);

    } while (option == 1);

    lines();
    std::cout << "Program ended. Thank you." << std::endl;

    return 0;
}

void lines() {
    std::cout << std::endl;
    for (int dash = 0; dash < 32; dash++) {
        std::cout << "-";
    }
    std::cout << std::endl;
}
```

```cpp
void input(double &firstNumber, double &secondNumber) {
    std::cout << "Enter the first number: ";
    std::cin >> firstNumber;
    std::cout << "Enter the second number: ";
    std::cin >> secondNumber;
}

void swapNumbers(double &firstNumber, double &secondNumber) {
    double temporaryVariable = firstNumber;
    firstNumber = secondNumber;
    secondNumber = temporaryVariable;
}

void prompt() {
    std::cout << "Do you want to swap again?" << std::endl;
    std::cout << "(1) Yes" << std::endl;
    std::cout << "(2) No" << std::endl << std::endl;
    std::cout << "Select option: ";
}
```

Sample Output:

Part 1

```
===== SWAPPING TWO NUMBERS =====

--------------------------------
Enter the first number: 15
Enter the second number: 30

        BEFORE SWAPPING
--------------------------------
First number   :  15
Second number  :  30

        AFTER SWAPPING
--------------------------------
First number   :  30
Second number  :  15

--------------------------------
Do you want to swap again?
(1) Yes
(2) No

Select option: 5

Invalid option. Please try again.

--------------------------------
Do you want to swap again?
```

Part 2

```
Select option: 1

--------------------------------
Enter the first number: -8
Enter the second number: 20

        BEFORE SWAPPING
--------------------------------
First number   :  -8
Second number  :  20

        AFTER SWAPPING
--------------------------------
First number   :  20
Second number  :  -8

--------------------------------
Do you want to swap again?
(1) Yes
(2) No

Select option: 9

Invalid option. Please try again.
```

Part 3

```
--------------------------------
Do you want to swap again?
(1) Yes
(2) No

Select option: 1

--------------------------------
Enter the first number: 12.75
Enter the second number: 4.25

        BEFORE SWAPPING
--------------------------------
First number   :  12.75
Second number  :  4.25

        AFTER SWAPPING
--------------------------------
First number   :  4.25
Second number  :  12.75

--------------------------------
Do you want to swap again?
(1) Yes
(2) No

Select option: 0

Invalid option. Please try again.

--------------------------------
Do you want to swap again?
(1) Yes
```

Part 4

```
(2) No

Select option: 1

--------------------------------
Enter the first number: 100
Enter the second number: 100

        BEFORE SWAPPING
--------------------------------
First number   :  100
Second number  :  100

        AFTER SWAPPING
--------------------------------
First number   :  100
Second number  :  100

--------------------------------
Do you want to swap again?
(1) Yes
(2) No

Select option: 2

--------------------------------
Program ended. Thank you.

--------------------------------
Process exited after 44.67 seconds w
urn value 0
Press any key to continue . . .
```

Code Analysis:

The code analysis shows that the program was created to swap two numbers by using different functions to make the code more organized and easier to understand. The important part of the code is the ***swapNumbers()*** function because it uses reference parameters with ***double &firstNumber*** and ***double &secondNumber***, which allows the original values from ***main()*** to be changed directly without needing to return a

value. The swapping process uses a temporary variable called ***temporaryVariable*** to store the first number before replacing the values, making sure that no data will be lost while switching the numbers. The ***input()*** function is also important because it handles the user input, while the ***lines()*** and ***prompt()*** functions help improve the program design and make the output cleaner. The use of ***double*** variables allows the program to swap both whole numbers and decimal numbers, making it more flexible. The output section uses ***std::setw()*** and ***std::left*** from the ***<iomanip>*** library to properly arrange the before and after swapping results.

During the creation of the code, some difficulties were faced such as understanding how reference variables work, because changing the value inside the function also changes the original variable in ***main()***. Another difficulty was making sure that the temporary variable was placed correctly in the swapping process because wrong placement can cause the original value to be overwritten. Handling the repeat option using ***do-while*** loops was also a little challenging because the condition needs to correctly determine if the program should continue or end.

Sections	Answer
Header File Declaration Section	`#include <iostream>` `#include <iomanip>`

Global Declaration Section	`void lines();`
	`void input(double &firstNumber, double &secondNumber);`
	`void swapNumbers(double &firstNumber, double &secondNumber);`
	`void prompt();`

Class Declaration and Method Definition Section	**None**

Main Function

```cpp
int main() {

    double firstNumber, secondNumber;
    int option = 0;

    std::cout << "===== SWAPPING TWO NUMBERS =====" << std::endl;

    do {
        lines();
        input(firstNumber, secondNumber);

        std::cout << std::endl << "\tBEFORE SWAPPING";
        lines();
        std::cout << std::left;
        std::cout << std::setw(15) << "First number"  << ":  " << firstNumber << std::endl;
        std::cout << std::setw(15) << "Second number" << ":  " << secondNumber << std::endl;

        swapNumbers(firstNumber, secondNumber);

        std::cout << std::endl << "\tAFTER SWAPPING";
        lines();
        std::cout << std::left;
        std::cout << std::setw(15) << "First number"  << ":  " << firstNumber << std::endl;
        std::cout << std::setw(15) << "Second number" << ":  " << secondNumber << std::endl;

        do {
            lines();
            prompt();
            std::cin >> option;

            if (option != 1 && option != 2) {
                std::cout << std::endl << "Invalid option. Please try again." << std::endl;
            }

        } while (option != 1 && option != 2);

    } while (option == 1);

    lines();
    std::cout << "Program ended. Thank you." << std::endl;

    return 0;
}
```

Method Definition

```cpp
void lines() {
    std::cout << std::endl;
    for (int dash = 0; dash < 32; dash++) {
        std::cout << "-";
    }
    std::cout << std::endl;
}

void input(double &firstNumber, double &secondNumber) {
    std::cout << "Enter the first number: ";
    std::cin >> firstNumber;
    std::cout << "Enter the second number: ";
    std::cin >> secondNumber;
}

void swapNumbers(double &firstNumber, double &secondNumber) {
    double temporaryVariable = firstNumber;
    firstNumber = secondNumber;
    secondNumber = temporaryVariable;
}

void prompt() {
    std::cout << "Do you want to swap again?" << std::endl;
    std::cout << "(1) Yes" << std::endl;
    std::cout << "(2) No" << std::endl << std::endl;
    std::cout << "Select option: ";
}
```

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

My Dev-C++ Code:

```cpp
#include <iostream>
#include <iomanip>

void lines();
void input(double &kelvinInput);
double kelvinToFahrenheit(double kelvinInput);
void prompt();

int main() {

    double kelvinInput;
    double fahrenheit = 0;
    int option = 0;

    std::cout << "======= KELVIN TO FAHRENHEIT CONVERTER =======" << std::endl;
    lines();
    std::cout << "\t\t  FORMULA:" << std::endl << std::endl;
    std::cout << "Fahrenheit = (Kelvin - 273.15) * 9 / 5 + 32" << std::endl;
    std::cout << std::fixed << std::setprecision(2);

    do {
        lines();
        input(kelvinInput);
        fahrenheit = kelvinToFahrenheit(kelvinInput);
        std::cout << "Temperature in Fahrenheit: " << fahrenheit << " " << char(248) << "F" << std::endl;

        do {
            lines();
            prompt();
            std::cin >> option;

            if (option != 1 && option != 2) {
                std::cout << std::endl << "Invalid option. Please try again." << std::endl;
            }

        } while (option != 1 && option != 2);

    } while (option == 1);

    lines();
    std::cout << "Program ended. Thank you." << std::endl;

    return 0;
}

void lines() {
    std::cout << std::endl;
    for (int dash = 0; dash < 46; dash++) {
        std::cout << "-";
    }
    std::cout << std::endl;
}

void input(double &kelvinInput) {
    do {
        std::cout << "Enter temperature in Kelvin: ";
        std::cin >> kelvinInput;

        if (kelvinInput < 0) {
            std::cout << "Invalid input. Kelvin temperature cannot be negative." << std::endl;
```

```
            lines();
        }

    } while (kelvinInput < 0);
}

double kelvinToFahrenheit(double kelvinInput) {
    double fahrenheit = (kelvinInput - 273.15) * 9 / 5 + 32;
    return fahrenheit;
}

void prompt() {
    std::cout << "Do you want to convert again?" << std::endl;
    std::cout << "(1) Yes" << std::endl;
    std::cout << "(2) No" << std::endl;
    std::cout << "Select option: ";
}
```

Sample Output:

Part 1:

```
=========== KELVIN TO FAHRENHEIT CONVERTER ===========

------------------------------------------------------------
                    FORMULA:

Fahrenheit = (Kelvin - 273.15) * 9 / 5 + 32

------------------------------------------------------------
Enter temperature in Kelvin: -25
Invalid input. Kelvin temperature cannot be negative.

------------------------------------------------------------
Enter temperature in Kelvin: 273.15
Temperature in Fahrenheit: 32.00 °F

------------------------------------------------------------
Do you want to convert again?
(1) Yes
(2) No

Select option: 5

Invalid option. Please try again.

------------------------------------------------------------
Do you want to convert again?
(1) Yes
(2) No

Select option: 1

------------------------------------------------------------
Enter temperature in Kelvin: 0
Temperature in Fahrenheit: -459.67 °F

------------------------------------------------------------
Do you want to convert again?
(1) Yes
(2) No

Select option: 3

Invalid option. Please try again.

------------------------------------------------------------
Do you want to convert again?
(1) Yes
(2) No
```

Part 2:

```
Select option: 1

------------------------------------------------------------
Enter temperature in Kelvin: -100
Invalid input. Kelvin temperature cannot be negative.

------------------------------------------------------------
Enter temperature in Kelvin: 310.15
Temperature in Fahrenheit: 98.60 °F

------------------------------------------------------------
Do you want to convert again?
(1) Yes
(2) No

Select option: 0

Invalid option. Please try again.

------------------------------------------------------------
Do you want to convert again?
(1) Yes
(2) No

Select option: 1

------------------------------------------------------------
Enter temperature in Kelvin: 373.15
Temperature in Fahrenheit: 212.00 °F

------------------------------------------------------------
Do you want to convert again?
(1) Yes
(2) No

Select option: 2

------------------------------------------------------------
Program ended. Thank you.

--------------------------------
Process exited after 67.21 seconds with return value 0
Press any key to continue . . .
```

Code Analysis:

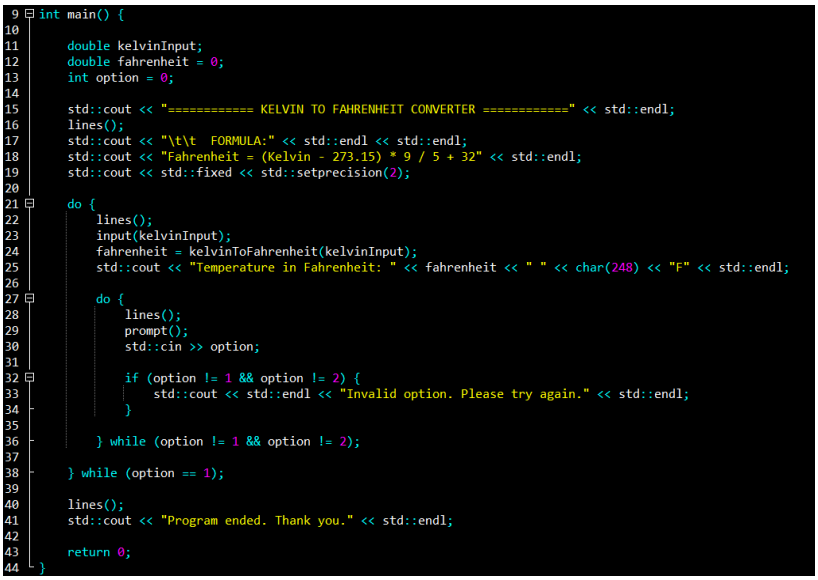
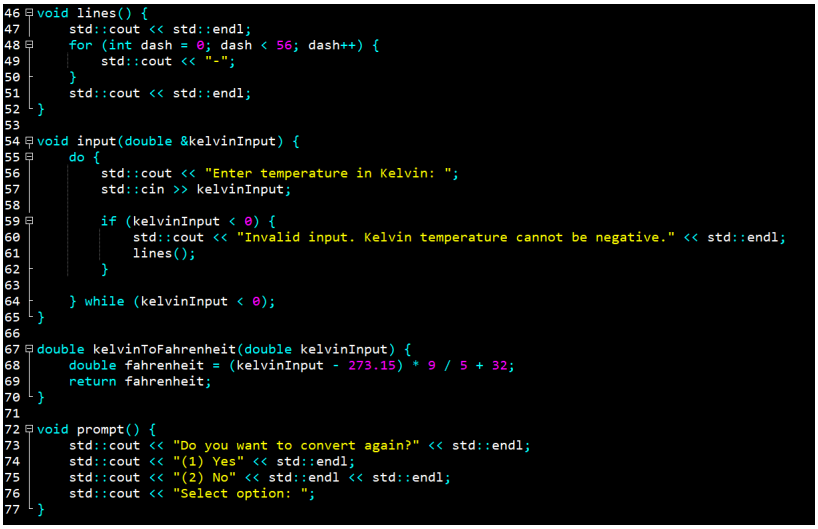
The code analysis shows that the program was created to convert temperature from Kelvin to Fahrenheit by using a separate function named ***kelvinToFahrenheit()***. The important part of the code is the formula ***(kelvinInput - 273.15) * 9 / 5 + 32*** inside the function because it is the main calculation that changes the Kelvin value into Fahrenheit. The program also uses function prototypes such as ***lines()***, ***input(double &kelvinInput)***, ***kelvinToFahrenheit(double kelvinInput)***, and ***prompt()*** to make the code more organized and easier to understand. The ***main()*** function handles the overall flow by getting the user input, calling the conversion function, and displaying the Fahrenheit result with ***setprecision(2)*** for a cleaner output.

One difficulty faced while making the code was understanding how to properly separate the tasks into different functions while still making sure the values can be passed and returned correctly. Another difficulty was applying the correct Kelvin to Fahrenheit formula because a small mistake in the order of operations can give a wrong answer. Handling invalid inputs was also a challenge because the ***input()*** function needed a ***do-while*** loop to prevent negative Kelvin values from being accepted. The repeat conversion option was also a bit difficult because the nested ***do-while*** loop needed to work properly so the program can ask again or end depending on the user's choice.

Sections	Answer
Header File Declaration Section	`#include <iostream>` `#include <iomanip>`

Global Declaration Section	`void lines();`
	`void input(double &kelvinInput);`
	`double kelvinToFahrenheit(double kelvinInput);`
	`void prompt();`

Class Declaration and Method Definition Section	**None**

Main Function	
Method Definition	

```cpp
int main() {

    double kelvinInput;
    double fahrenheit = 0;
    int option = 0;

    std::cout << "============ KELVIN TO FAHRENHEIT CONVERTER ============" << std::endl;
    lines();
    std::cout << "\t\t  FORMULA:" << std::endl << std::endl;
    std::cout << "Fahrenheit = (Kelvin - 273.15) * 9 / 5 + 32" << std::endl;
    std::cout << std::fixed << std::setprecision(2);

    do {
        lines();
        input(kelvinInput);
        fahrenheit = kelvinToFahrenheit(kelvinInput);
        std::cout << "Temperature in Fahrenheit: " << fahrenheit << " " << char(248) << "F" << std::endl;

        do {
            lines();
            prompt();
            std::cin >> option;

            if (option != 1 && option != 2) {
                std::cout << std::endl << "Invalid option. Please try again." << std::endl;
            }

        } while (option != 1 && option != 2);

    } while (option == 1);

    lines();
    std::cout << "Program ended. Thank you." << std::endl;

    return 0;
}
```

```cpp
void lines() {
    std::cout << std::endl;
    for (int dash = 0; dash < 56; dash++) {
        std::cout << "-";
    }
    std::cout << std::endl;
}

void input(double &kelvinInput) {
    do {
        std::cout << "Enter temperature in Kelvin: ";
        std::cin >> kelvinInput;

        if (kelvinInput < 0) {
            std::cout << "Invalid input. Kelvin temperature cannot be negative." << std::endl;
            lines();
        }

    } while (kelvinInput < 0);
}

double kelvinToFahrenheit(double kelvinInput) {
    double fahrenheit = (kelvinInput - 273.15) * 9 / 5 + 32;
    return fahrenheit;
}

void prompt() {
    std::cout << "Do you want to convert again?" << std::endl;
    std::cout << "(1) Yes" << std::endl;
    std::cout << "(2) No" << std::endl << std::endl;
    std::cout << "Select option: ";
}
```

3. Create a C++ program that has a function that will calculate the distance between two points.

My Dev-C++ Code:

```cpp
#include <iostream>
#include <iomanip>
#include <cmath>

void lines();
void input(double &x1, double &y1, double &x2, double &y2);
double calculateDistance(double x1, double y1, double x2, double y2);
void prompt();

int main() {

    double x1, y1, x2, y2;
    double distance = 0;
    int option = 0;

    std::cout << "========= DISTANCE BETWEEN TWO POINTS ========" << std::endl;
    lines();
    std::cout << "\t\t  FORMULA:" << std::endl << std::endl;
    std::cout << "Distance = sqrt[(x2 - x1)^2 + (y2 - y1)^2]" << std::endl;
    std::cout << std::fixed << std::setprecision(2);

    do {
        lines();
        input(x1, y1, x2, y2);
        distance = calculateDistance(x1, y1, x2, y2);
        std::cout << "The distance between the two points is " << distance << std::endl;

        do {
            lines();
            prompt();
            std::cin >> option;

            if (option != 1 && option != 2) {
                std::cout << std::endl << "Invalid option. Please try again." << std::endl;
            }

        } while (option != 1 && option != 2);

    } while (option == 1);

    lines();
    std::cout << "Program ended. Thank you." << std::endl;

    return 0;
}

void lines() {
    std::cout << std::endl;
    for (int dash = 0; dash < 46; dash++) {
        std::cout << "-";
    }
    std::cout << std::endl;
}

void input(double &x1, double &y1, double &x2, double &y2) {
    std::cout << "Enter the x-coordinate of the first point: ";
```

```
57      std::cin >> x1;
58      std::cout << "Enter the y-coordinate of the first point: ";
59      std::cin >> y1;
60      std::cout << "Enter the x-coordinate of the second point: ";
61      std::cin >> x2;
62      std::cout << "Enter the y-coordinate of the second point: ";
63      std::cin >> y2;
64  }
65
66  double calculateDistance(double x1, double y1, double x2, double y2) {
67      double distance;
68      distance = sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
69      return distance;
70  }
71
72  void prompt() {
73      std::cout << "Do you want to compute again?" << std::endl;
74      std::cout << "(1) Yes" << std::endl;
75      std::cout << "(2) No" << std::endl << std::endl;
76      std::cout << "Select option: ";
77  }
```

Sample Output:

Part 1:

```
========== DISTANCE BETWEEN TWO POINTS ==========

------------------------------------------------
                  FORMULA:

Distance = sqrt[(x2 - x1)^2 + (y2 - y1)^2]

------------------------------------------------
Enter the x-coordinate of the first point: 0
Enter the y-coordinate of the first point: 0
Enter the x-coordinate of the second point: 3
Enter the y-coordinate of the second point: 4
The distance between the two points is 5.00

------------------------------------------------
Do you want to compute again?
(1) Yes
(2) No

Select option: 5

Invalid option. Please try again.

------------------------------------------------
Do you want to compute again?
(1) Yes
(2) No

Select option: 1

------------------------------------------------
Enter the x-coordinate of the first point: 2
Enter the y-coordinate of the first point: 5
Enter the x-coordinate of the second point: 2
Enter the y-coordinate of the second point: 5
The distance between the two points is 0.00
```

Part 2:

```
------------------------------------------------
Do you want to compute again?
(1) Yes
(2) No

Select option: 0

Invalid option. Please try again.

------------------------------------------------
Do you want to compute again?
(1) Yes
(2) No

Select option: 1

------------------------------------------------
Enter the x-coordinate of the first point: -1
Enter the y-coordinate of the first point: -2
Enter the x-coordinate of the second point: 4
Enter the y-coordinate of the second point: 2
The distance between the two points is 6.40

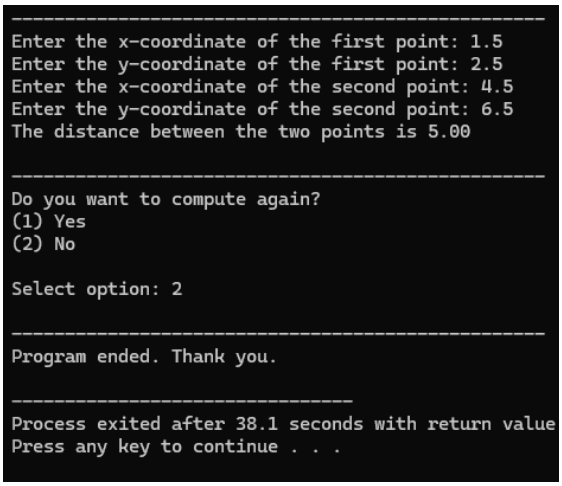
------------------------------------------------
Do you want to compute again?
(1) Yes
(2) No

Select option: 8

Invalid option. Please try again.

------------------------------------------------
Do you want to compute again?
(1) Yes
(2) No
```

Part 3:



Code Analysis:

The program was created to solve the problem of calculating the distance between two points using the C++ programming language. It used different functions to make the code more organized, especially the important function ***double calculateDistance(double x1, double y1, double x2, double y2);*** because this part contains the main formula ***sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2))*** that computes the distance between the given points. The function prototype before the ***main()*** function was also important because it allows the program to recognize the functions before they are called. The ***input()*** function was used to get the values of the coordinates, while the use of reference variables with ***&*** helped in passing the entered values correctly.

The code also used ***#include <cmath>*** for the ***sqrt()*** and ***pow()*** functions and use iomanip with ***setprecision(2)*** to make the displayed answer more clean and easier to read. During the development of the program, some difficulties were faced, especially in understanding how to apply the mathematical distance formula into C++ syntax and knowing when to use functions like ***sqrt()*** and ***pow()***. Another difficulty was arranging the functions properly because each part of the code needs to connect with each other without causing errors. The loop using ***do-while*** was also a bit challenging because the condition must allow the program to repeat only when the selected option is ***1***, while ending correctly when the option is ***2***.

There were also small challenges in checking the format of the output and making sure the distance result was displayed with the correct decimal places. After testing and correcting the errors, the program was able to work properly and showed a better understanding of using functions, parameters, loops, and mathematical operations in C++ to solve a given problem.

T.I.P TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

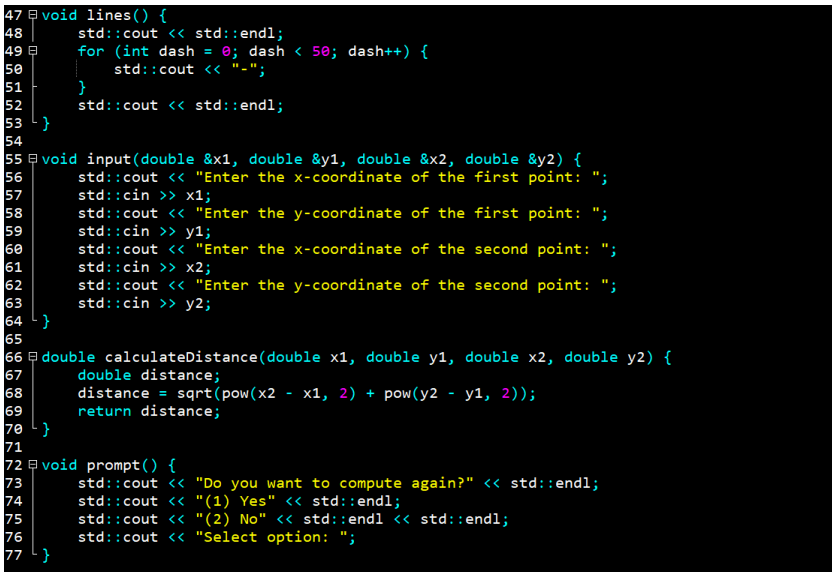
Sections	Answer

Header File Declaration Section	`#include <iostream>`
	`#include <iomanip>`
	`#include <cmath>`

Global Declaration Section	`void lines();`
	`void input(double &x1, double &y1, double &x2, double &y2);`
	`double calculateDistance(double x1, double y1, double x2, double y2);`
	`void prompt();`

Class Declaration and Method Definition Section	**None**

Main Function	`int main() {`
	`  double x1, y1, x2, y2;`
	`  double distance = 0;`
	`  int option = 0;`
	
	`  std::cout << "========== DISTANCE BETWEEN TWO POINTS ==========" << std::endl;`
	`  lines();`
	`  std::cout << "\t\t  FORMULA:" << std::endl << std::endl;`
	`  std::cout << "Distance = sqrt[(x2 - x1)^2 + (y2 - y1)^2]" << std::endl;`
	`  std::cout << std::fixed << std::setprecision(2);`
	
	`  do {`
	`  lines();`
	`  input(x1, y1, x2, y2);`
	`  distance = calculateDistance(x1, y1, x2, y2);`
	`  std::cout << "The distance between the two points is " << distance << std::endl;`
	
	`  do {`
	`  lines();`
	`  prompt();`
	`  std::cin >> option;`
	
	`  if (option != 1 && option != 2) {`
	`  std::cout << std::endl << "Invalid option. Please try again." << std::endl;`
	`  }`
	
	`  } while (option != 1 && option != 2);`
	
	`  } while (option == 1);`
	
	`  lines();`
	`  std::cout << "Program ended. Thank you." << std::endl;`
	
	`  return 0;`
	`}`

Method Definition	

```
void lines() {
    std::cout << std::endl;
    for (int dash = 0; dash < 50; dash++) {
        std::cout << "-";
    }
    std::cout << std::endl;
}

void input(double &x1, double &y1, double &x2, double &y2) {
    std::cout << "Enter the x-coordinate of the first point: ";
    std::cin >> x1;
    std::cout << "Enter the y-coordinate of the first point: ";
    std::cin >> y1;
    std::cout << "Enter the x-coordinate of the second point: ";
    std::cin >> x2;
    std::cout << "Enter the y-coordinate of the second point: ";
    std::cin >> y2;
}

double calculateDistance(double x1, double y1, double x2, double y2) {
    double distance;
    distance = sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
    return distance;
}

void prompt() {
    std::cout << "Do you want to compute again?" << std::endl;
    std::cout << "(1) Yes" << std::endl;
    std::cout << "(2) No" << std::endl << std::endl;
    std::cout << "Select option: ";
}
```

4. Modify the code given in ILO B and add the following functions:

   a) A function to compute for the area of a triangle
   b) A function to compute for the perimeter of a triangle
   c) A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

My Dev-C++ Code:

```
#include <iostream>
#include <iomanip>
#include <cmath>
#include <string>

const double PI = 3.14159265358979323846;

class Triangle {
private:
    double totalAngle;
    double angleA, angleB, angleC;
    double sideA, sideB, sideC;

    double computeSide(double knownSide, double knownAngle, double targetAngle);

public:
    Triangle(double A, double B, double C, double a);

    void setAngles(double A, double B, double C, double a);
    bool validateTriangle();

    double computeArea();
    double computePerimeter();
    std::string determineType();
};

void lines();
void menu();
void formulas();
void inputs(double &A, double &B, double &C, double &sideA);
```

```cpp

int main() {

    double angleA, angleB, angleC;
    double sideA;
    int option = 0;

    std::cout << "====================== TRIANGLE CALCULATOR ======================" << std::endl;
    lines();
    formulas();
    std::cout << std::fixed << std::setprecision(2);

    while (option != 4) {
        lines();
        menu();
        std::cin >> option;

        switch (option) {
            case 1:
            case 2:
            case 3: {
                lines();
                inputs(angleA, angleB, angleC, sideA);
                Triangle triangle(angleA, angleB, angleC, sideA);

                if (triangle.validateTriangle()) {
                    std::cout << std::endl << "The shape is a VALID triangle." << std::endl;

                    switch (option) {
                        case 1:
                            std::cout << "Area of Triangle: " << triangle.computeArea() << std::endl;
                            break;
                        case 2:
                            std::cout << "Perimeter of Triangle: " << triangle.computePerimeter() << std::endl;
                            break;
                        case 3:
                            std::cout << "Triangle Type: " << triangle.determineType() << std::endl;
                            break;
                    }
                } else {
                    std::cout << std::endl << "The shape is NOT A VALID triangle." << std::endl;
                }

                break;
            }

            case 4:
                lines();
                std::cout << "Program ended. Thank you." << std::endl;
                break;

            default:
                std::cout << std::endl << "Invalid option. Please try again." << std::endl;
        }
    }

    return 0;
}

void lines() {
    std::cout << std::endl;
    for (int i = 0; i < 66; i++) {
        std::cout << "-";
    }
    std::cout << std::endl;
}

void menu() {
    std::cout << "(1) Compute the Area" << std::endl;
    std::cout << "(2) Compute the Perimeter" << std::endl;
    std::cout << "(3) Determine Triangle Type" << std::endl;
    std::cout << "(4) Exit" << std::endl << std::endl;
    std::cout << "Select option: ";
}

```

```cpp
void formulas() {
    std::cout << "\t\t\tFORMULAS AND GUIDES";
    lines();
    std::cout << "AREA OF TRIANGLE:" << std::endl;
    std::cout << char(249) << " A = 1/2 (b)(c) sin(A)" << std::endl << std::endl;
    std::cout << "PERIMETER OF TRIANGLE:" << std::endl;
    std::cout << char(249) << " P = a + b + c" << std::endl << std::endl;
    std::cout << "TRIANGLE TYPE:" << std::endl;
    std::cout << char(249) << " Acute-angled Triangle  : All angles are less than 90 degrees" << std::endl;
    std::cout << char(249) << " Obtuse-angled Triangle : One angle is greater than 90 degrees" << std::endl;
    std::cout << char(249) << " Others                 : One angle is exactly 90 degrees" << std::endl;
}

void inputs(double &A, double &B, double &C, double &sideA) {
    std::cout << "Enter Angle A: ";
    std::cin >> A;
    std::cout << "Enter Angle B: ";
    std::cin >> B;
    std::cout << "Enter Angle C: ";
    std::cin >> C;
    std::cout << "Enter Side A: ";
    std::cin >> sideA;
}
```

```cpp

Triangle::Triangle(double A, double B, double C, double a) {
    setAngles(A, B, C, a);
}

void Triangle::setAngles(double A, double B, double C, double a) {
    angleA = A;
    angleB = B;
    angleC = C;
    sideA = a;

    totalAngle = A + B + C;

    if (validateTriangle()) {
        sideB = computeSide(sideA, angleA, angleB);
        sideC = computeSide(sideA, angleA, angleC);
    }
}

bool Triangle::validateTriangle() {
    return (fabs(totalAngle - 180.0) < 0.0001 &&
            angleA > 0 &&
            angleB > 0 &&
            angleC > 0 &&
            sideA > 0);
}

double Triangle::computeSide(double knownSide, double knownAngle, double targetAngle) {
    return knownSide * sin(targetAngle * PI / 180) / sin(knownAngle * PI / 180);
}

```

```cpp
double Triangle::computeArea() {
    return 0.5 * sideB * sideC * sin(angleA * PI / 180);
}

double Triangle::computePerimeter() {
    return sideA + sideB + sideC;
}

std::string Triangle::determineType() {
    if (angleA < 90 && angleB < 90 && angleC < 90) {
        return "Acute-angled Triangle";
    } else if (angleA > 90 || angleB > 90 || angleC > 90) {
        return "Obtuse-angled Triangle";
    } else {
        return "Others";
    }
}
```

Sample Output:

```
====================== TRIANGLE CALCULATOR ======================

------------------------------------------------------------------
                        FORMULAS AND GUIDES
------------------------------------------------------------------
AREA OF TRIANGLE:
• A = 1/2 (b)(c) sin(A)

PERIMETER OF TRIANGLE:
• P = a + b + c

TRIANGLE TYPE:
• Acute-angled Triangle  : All angles are less than 90 degrees
• Obtuse-angled Triangle : One angle is greater than 90 degrees
• Others                 : One angle is exactly 90 degrees

------------------------------------------------------------------
(1) Compute the Area
(2) Compute the Perimeter
(3) Determine Triangle Type
(4) Exit

Select option: 7

Invalid option. Please try again.

------------------------------------------------------------------
(1) Compute the Area
(2) Compute the Perimeter
(3) Determine Triangle Type
(4) Exit

Select option: 1

------------------------------------------------------------------
Enter Angle A: 50
Enter Angle B: 50
Enter Angle C: 60
Enter Side A: -10

The shape is NOT A VALID triangle.
```

```
------------------------------------------------------------------
(1) Compute the Area
(2) Compute the Perimeter
(3) Determine Triangle Type
(4) Exit

Select option: 2

------------------------------------------------------------------
Enter Angle A: 50
Enter Angle B: 50
Enter Angle C: 70
Enter Side A: 8

The shape is NOT A VALID triangle.

------------------------------------------------------------------
(1) Compute the Area
(2) Compute the Perimeter
(3) Determine Triangle Type
(4) Exit

Select option: 1

------------------------------------------------------------------
Enter Angle A: 60
Enter Angle B: 60
Enter Angle C: 60
Enter Side A: 10

The shape is a VALID triangle.
Area of Triangle: 43.30

------------------------------------------------------------------
(1) Compute the Area
(2) Compute the Perimeter
(3) Determine Triangle Type
(4) Exit

Select option: 2

------------------------------------------------------------------
Enter Angle A: 60
Enter Angle B: 60
```

```
Enter Angle C: 60
Enter Side A: 10

The shape is a VALID triangle.
Perimeter of Triangle: 30.00

------------------------------------------------------------------
(1) Compute the Area
(2) Compute the Perimeter
(3) Determine Triangle Type
(4) Exit

Select option: 3

------------------------------------------------------------------
Enter Angle A: 100
Enter Angle B: 40
Enter Angle C: 40
Enter Side A: 10

The shape is a VALID triangle.
Triangle Type: Obtuse-angled Triangle

------------------------------------------------------------------
(1) Compute the Area
(2) Compute the Perimeter
(3) Determine Triangle Type
(4) Exit

Select option: 3

------------------------------------------------------------------
Enter Angle A: 90
Enter Angle B: 45
Enter Angle C: 45
Enter Side A: 8

The shape is a VALID triangle.
Triangle Type: Others

------------------------------------------------------------------
(1) Compute the Area
(2) Compute the Perimeter
(3) Determine Triangle Type
```

```
(4) Exit

Select option: 4

--------------------------------------------------------------------
Program ended. Thank you.

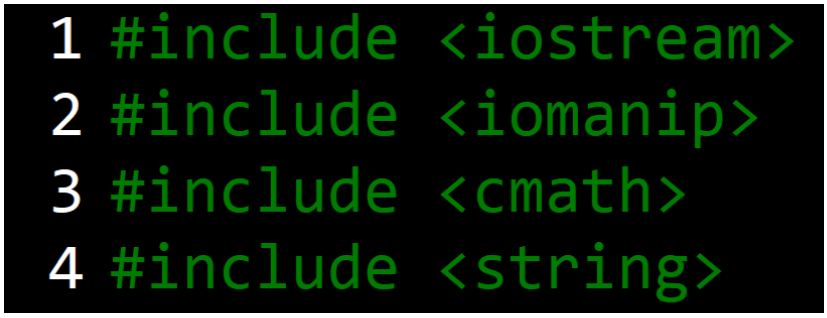
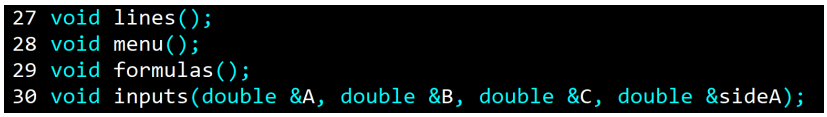
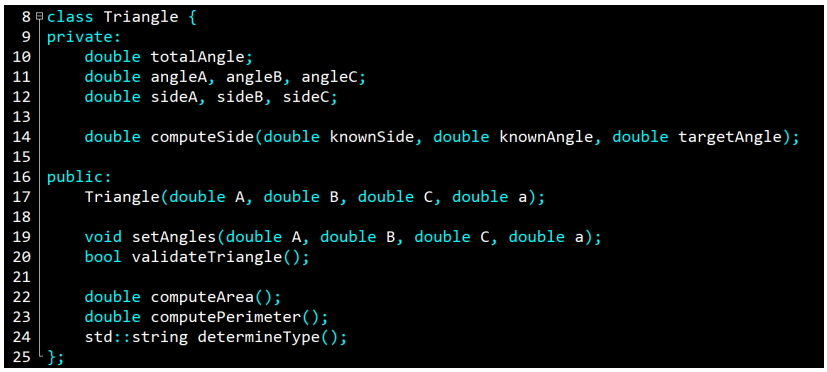
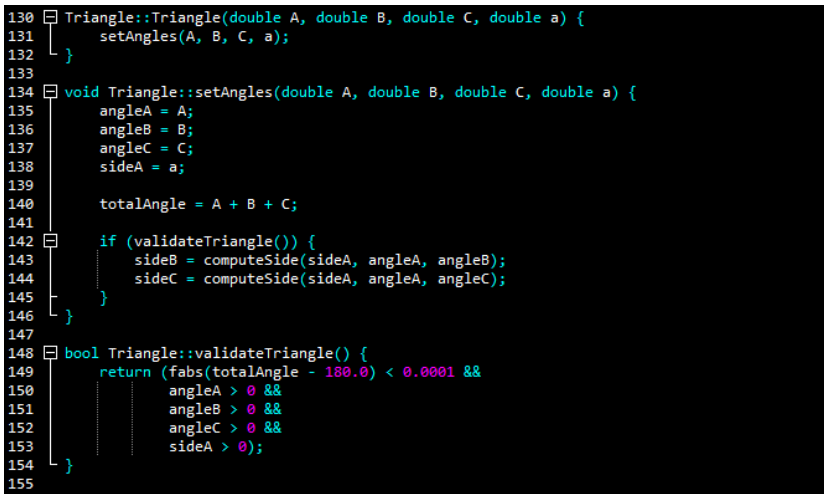
--------------------------------
Process exited after 93.3 seconds with return value 0
Press any key to continue . . .
```

Code Analysis:

The code analysis shows that the program was improved by modifying the original Triangle class and adding more functions to solve different triangle problems. The code started by adding important variables like ***sideA***, ***sideB***, and ***sideC*** inside the class to store the side measurements needed for the calculations. The constructor ***Triangle(double A, double B, double C, double a)*** was changed so it can accept the three angles and one side of the triangle, then the function ***setAngles()*** was used to assign the values and calculate the ***totalAngle*** using ***A + B + C***. The ***validateTriangle()*** function became an important improvement because instead of only checking if the total angle is less than or equal to 180 like the original code, it now uses ***fabs(totalAngle - 180.0) < 0.0001*** to make sure the triangle has an exact total angle of 180 degrees while also checking if all angles and side values are positive.

The added function ***computeSide()*** was also important because it uses the Law of Sines formula to calculate the missing sides of the triangle, which helps the program get the needed values for the area and perimeter calculations. The ***computeArea()*** function uses the formula ***0.5 * sideB * sideC * sin(angleA * PI / 180)*** to calculate the area of the triangle, while the ***computePerimeter()*** function uses ***sideA + sideB + sideC*** to find the total perimeter. The ***determineType()*** function was added to identify the triangle category by checking the angles using conditions, allowing the program to display if the triangle is acute-angled, obtuse-angled, or others. The menu system using the ***while*** loop and ***switch*** statement also helped organize the program because it allows the user to choose between different operations like area, perimeter, and triangle type.

During the coding process, some difficulties were faced such as understanding how trigonometric functions work in C++ because the ***sin()*** function needs radians instead of degrees, so the conversion using ***angleA * PI / 180*** was needed. Another difficulty was applying the Law of Sines correctly in the ***computeSide()*** function because finding the missing sides required understanding the relationship between angles and opposite sides. There was also a challenge in improving the triangle validation because the original condition ***totalAngle <= 180*** could still accept invalid triangles, so a more accurate condition was created. Handling the class structure and making sure every function works together was also challenging because changing one part of the code could affect the other calculations.

Sections	Answer
Header File Declaration Section	
Global Declaration Section	
Class Declaration and Method Definition Section	
	

```
#include <iostream>
#include <iomanip>
#include <cmath>
#include <string>
```

```
void lines();
void menu();
void formulas();
void inputs(double &A, double &B, double &C, double &sideA);
```

```
class Triangle {
private:
    double totalAngle;
    double angleA, angleB, angleC;
    double sideA, sideB, sideC;

    double computeSide(double knownSide, double knownAngle, double targetAngle);

public:
    Triangle(double A, double B, double C, double a);

    void setAngles(double A, double B, double C, double a);
    bool validateTriangle();

    double computeArea();
    double computePerimeter();
    std::string determineType();
};
```

```
Triangle::Triangle(double A, double B, double C, double a) {
    setAngles(A, B, C, a);
}

void Triangle::setAngles(double A, double B, double C, double a) {
    angleA = A;
    angleB = B;
    angleC = C;
    sideA = a;

    totalAngle = A + B + C;

    if (validateTriangle()) {
        sideB = computeSide(sideA, angleA, angleB);
        sideC = computeSide(sideA, angleA, angleC);
    }
}

bool Triangle::validateTriangle() {
    return (fabs(totalAngle - 180.0) < 0.0001 &&
            angleA > 0 &&
            angleB > 0 &&
            angleC > 0 &&
            sideA > 0);
}
```

```cpp
double Triangle::computeSide(double knownSide, double knownAngle, double targetAngle) {
    return knownSide * sin(targetAngle * PI / 180) / sin(knownAngle * PI / 180);
}

double Triangle::computeArea() {
    return 0.5 * sideB * sideC * sin(angleA * PI / 180);
}

double Triangle::computePerimeter() {
    return sideA + sideB + sideC;
}

std::string Triangle::determineType() {
    if (angleA < 90 && angleB < 90 && angleC < 90) {
        return "Acute-angled Triangle";
    } else if (angleA > 90 || angleB > 90 || angleC > 90) {
        return "Obtuse-angled Triangle";
    } else {
        return "Others";
    }
}
```

Main Function

```cpp
int main() {

    double angleA, angleB, angleC;
    double sideA;
    int option = 0;

    std::cout << "====================== TRIANGLE CALCULATOR ======================" << std::endl;
    lines();
    formulas();
    std::cout << std::fixed << std::setprecision(2);

    while (option != 4) {
        lines();
        menu();
        std::cin >> option;

        switch (option) {
            case 1:
            case 2:
            case 3: {
                lines();
                inputs(angleA, angleB, angleC, sideA);
                Triangle triangle(angleA, angleB, angleC, sideA);

                if (triangle.validateTriangle()) {
                    std::cout << std::endl << "The shape is a VALID triangle." << std::endl;

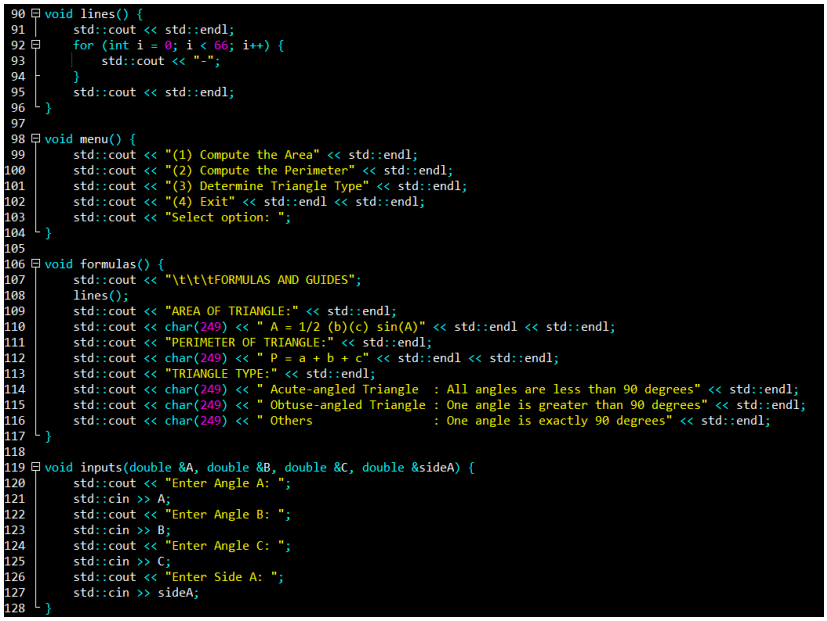
                    switch (option) {
                        case 1:
                            std::cout << "Area of Triangle: " << triangle.computeArea() << std::endl;
                            break;
                        case 2:
                            std::cout << "Perimeter of Triangle: " << triangle.computePerimeter() << std::endl;
                            break;
                        case 3:
                            std::cout << "Triangle Type: " << triangle.determineType() << std::endl;
                            break;
                    }
                } else {
                    std::cout << std::endl << "The shape is NOT A VALID triangle." << std::endl;
                }

                break;
            }

            case 4:
                lines();
                std::cout << "Program ended. Thank you." << std::endl;
                break;

            default:
                std::cout << std::endl << "Invalid option. Please try again." << std::endl;
        }
    }

    return 0;
}
```

Method Definition	

```cpp
void lines() {
    std::cout << std::endl;
    for (int i = 0; i < 66; i++) {
        std::cout << "-";
    }
    std::cout << std::endl;
}

void menu() {
    std::cout << "(1) Compute the Area" << std::endl;
    std::cout << "(2) Compute the Perimeter" << std::endl;
    std::cout << "(3) Determine Triangle Type" << std::endl;
    std::cout << "(4) Exit" << std::endl << std::endl;
    std::cout << "Select option: ";
}

void formulas() {
    std::cout << "\t\t\tFORMULAS AND GUIDES";
    lines();
    std::cout << "AREA OF TRIANGLE:" << std::endl;
    std::cout << char(249) << " A = 1/2 (b)(c) sin(A)" << std::endl << std::endl;
    std::cout << "PERIMETER OF TRIANGLE:" << std::endl;
    std::cout << char(249) << " P = a + b + c" << std::endl << std::endl;
    std::cout << "TRIANGLE TYPE:" << std::endl;
    std::cout << char(249) << " Acute-angled Triangle  : All angles are less than 90 degrees" << std::endl;
    std::cout << char(249) << " Obtuse-angled Triangle : One angle is greater than 90 degrees" << std::endl;
    std::cout << char(249) << " Others                 : One angle is exactly 90 degrees" << std::endl;
}

void inputs(double &A, double &B, double &C, double &sideA) {
    std::cout << "Enter Angle A: ";
    std::cin >> A;
    std::cout << "Enter Angle B: ";
    std::cin >> B;
    std::cout << "Enter Angle C: ";
    std::cin >> C;
    std::cout << "Enter Side A: ";
    std::cin >> sideA;
}
```

Conclusion:

The activity helped in learning more about the basic concepts of C++ programming by making different programs using functions, classes, loops, conditions, and mathematical formulas. It also gave a better understanding of why function prototypes, methods, and classes are important because they make the code more organized and easier to follow. While doing the activities, there was also more learning about using different header files like <iostream>, <iomanip>, <cmath>, and <string> to solve different programming problems. Every activity also helped improve logical thinking and coding skills little by little.

The procedure was easy to follow because every step explained what should be done before making the code. It started with the basic C++ structure, then continued to using classes, objects, and different programming problems that became a little harder. Following the required format also helped in understanding where to place the function prototypes, the main() function, and the function definitions. There were some mistakes while coding, but after testing and checking the code many times, the errors were slowly fixed until the programs worked correctly.

The supplementary activities gave more practice in using the lessons that were learned from the previous activities. Making programs like swapping two numbers, converting Kelvin to Fahrenheit, calculating the distance between two points, and improving the Triangle class helped in understanding how functions and classes can be used in different situations. Some parts were a little confusing at first, especially when using mathematical formulas, trigonometric functions, reference parameters, and class methods, but after reading the code carefully and testing different inputs, there was a better understanding of how everything works together. These activities

also helped improve problem-solving skills and gave more confidence in writing C++ programs.

The activity was a good learning experience because it gave more practice in writing C++ programs and using different programming concepts together. There were many small challenges while making the codes, especially when fixing errors, connecting the functions, and making sure the outputs were correct, but those problems were solved by testing the programs many times and making small improvements. There is still a need to improve more in writing cleaner code, understanding object-oriented programming better, and solving more difficult programming problems. Even with those challenges, the activity helped build a stronger understanding of C++ programming and gave more confidence for the next programming activities.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **grammar correction, formatting, explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to explain how the code works by giving a detailed explanation on the different parts of the code. Moreover, the AI was used to assist in improving the grammar of the code analysis, allowing the explanation to be written more clearly and fluently in English.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.